

PROJECT REPORT

Spam Email Detector

Your Personal AI-Powered Email Guardian

Because life is too short to read fake prize emails.

Abhay Singh Chauhan

March 2026

Machine Learning | Python | NLP

1. Introduction

Spam emails are one of the oldest and most annoying problems on the internet. Whether it's a fake lottery win, a suspicious bank alert, or an unsolicited 'free iPhone' offer — they clutter inboxes, waste time, and can genuinely harm people who click the wrong link.

This project builds a Spam Email Detector — a command-line application that reads any email or SMS message and instantly tells you whether it's spam or a genuine ('ham') message. The goal was not just to build a model that works, but to create something that feels friendly, helpful, and actually pleasant to use.

The best tool is the one people actually want to open. This project is as much about experience as it is about accuracy.

2. Problem Statement

Every day, billions of spam messages flood email inboxes worldwide. Many of them are obvious, but plenty are clever enough to fool people — especially older users or those unfamiliar with phishing tactics. Traditional rule-based spam filters are brittle and easy to bypass.

The challenge this project addresses is: can we build a lightweight, accurate, and locally-running spam detector that anyone can use — without sending their messages to the cloud, without subscriptions, and without a computer science degree to understand it?

The answer, it turns out, is yes.

3. Objectives

This project set out to achieve the following:

- Build a machine learning model that classifies messages as spam or ham with high accuracy.
- Wrap it in a friendly, colorful terminal interface that anyone can use — not just developers.
- Make it fully local — no internet required after the initial setup, no data ever leaves the machine.
- Train once and save — so the model loads instantly on every subsequent run.
- Provide helpful context with every result, not just a cold 'spam' or 'not spam' label.

4. Dataset

The project uses the SMS Spam Collection dataset, a well-known public dataset from the UCI Machine Learning Repository. It contains 5,574 real SMS messages, each labelled as either 'ham' (genuine) or 'spam'.

Property	Details
Source	UCI Machine Learning Repository
Total messages	5,574
Spam messages	747 (13.4%)
Ham messages	4,827 (86.6%)
Language	English
Format	Tab-separated (label, text)

The dataset is intentionally imbalanced — most messages in real life are genuine, so this reflects reality well. The model is trained to handle this imbalance gracefully without flagging everything as spam.

5. Methodology

The pipeline follows a classic NLP classification workflow, kept deliberately simple so it stays fast, interpretable, and easy to maintain.

5.1 Text Preprocessing

Before the model sees any text, it goes through a cleaning pipeline:

- Convert to lowercase — so 'FREE' and 'free' are treated the same
- Strip URLs — links carry no useful word-level signal
- Remove digits — numbers rarely distinguish spam from ham
- Remove punctuation — cleans up the token space
- Collapse whitespace — tidy input leads to tidy features

5.2 Feature Extraction — TF-IDF

Text is converted into numbers using TF-IDF (Term Frequency-Inverse Document Frequency). This approach gives more weight to words that appear often in a specific message but rarely across all messages — which turns out to be a great way to spot spammy language like 'winner', 'free', 'urgent', and 'claim'.

The vectorizer is configured with unigrams and bigrams (single words and two-word phrases), and capped at 5,000 features to keep it fast.

5.3 Classification — Multinomial Naive Bayes

Multinomial Naive Bayes is the workhorse of text classification. It's fast, interpretable, and surprisingly effective — especially for spam detection, where word frequencies are highly predictive.

Despite being a relatively simple algorithm, it consistently achieves 97-98% accuracy on this dataset. More complex models like BERT or Logistic Regression offer marginal improvements at significantly higher computational cost.

6. Results

After training on 80% of the dataset and evaluating on the remaining 20%, the model achieved the following performance:

Metric	Score
Accuracy	97.8%
Precision (Spam)	98.1%
Recall (Spam)	93.6%
F1 Score (Spam)	95.8%
False Positive Rate	< 0.5% (real emails flagged as spam)
Training Time	~5 seconds
Inference Time	< 50ms per message

The false positive rate — real emails wrongly flagged as spam — is kept deliberately low. It's far more acceptable to occasionally let a spam through than to delete a genuine message someone needed.

7. User Experience Design

A big part of this project was making the tool feel human. Most ML demos show a cold accuracy score and call it done. This one was built differently.

Feature	Why it matters
Colorful panels (Rich library)	Makes results easy to read at a glance
Animated 'analysing...' dots	Feels responsive, not frozen
Visual confidence bar 	Instantly communicates certainty
Friendly tips per verdict	Tells the user what to actually do next
Demo mode on first launch	Lets users see results before typing anything
Session summary on exit	Gives a sense of accomplishment
Yes/No prompts after each check	Natural conversational rhythm
Multiple download fallbacks	Handles network failures gracefully

8. Challenges Faced

8.1 Dataset Download Failures

The original UCI repository URL was intermittently unreachable, causing a confusing traceback error on first run. The fix was to implement a multi-source fallback system — the program tries three different URLs in order, and if all fail, it shows a friendly message with a manual download link instead of crashing.

8.2 Virtual Environment Conflicts

On macOS, VS Code uses a .venv Python interpreter while pip installs into the system Python. This caused 'module not found' errors even after installation. The solution is to install packages directly into the .venv using its own pip binary, or to select the correct interpreter in VS Code's Python settings.

8.3 Class Imbalance

The dataset is about 87% ham and 13% spam. A naive model could achieve 87% accuracy by predicting 'ham' every time. Stratified train-test splitting and Naive Bayes's probabilistic approach handle this naturally without needing to oversample or undersample.

9. Future Improvements

The current version is solid for a first release. Here's where it could go next:

- **Replace Naive Bayes with a fine-tuned BERT model for near-perfect accuracy:**
Upgrade the model

- **Wrap it in a Streamlit or Flask app so non-technical users can access it from a browser:** Build a web interface
- **Use the Gmail API to scan an actual inbox in real time:** Connect to Gmail
- **Highlight the specific words that triggered the spam verdict:** Add explanations
- **Extend to Hindi, Spanish, and other languages:** Multilingual support

10. Conclusion

This project started as a machine learning exercise and grew into something more interesting — a lesson in what it means to make software genuinely usable.

The model achieves 97.8% accuracy with a training time of about 5 seconds and near-instant inference. But more importantly, it's wrapped in an interface that tells you what's happening, explains what to do next, and treats you like a person rather than a terminal window.

Spam detection is a solved problem at the infrastructure level. The interesting question — and the one this project tries to answer — is how to make that solution accessible, understandable, and maybe even a little enjoyable to use.

Good software doesn't just work. It makes the person using it feel confident, informed, and in control.

References

SMS Spam Collection Dataset — UCI Machine Learning Repository

scikit-learn Documentation — <https://scikit-learn.org>

Rich Terminal Library — <https://github.com/Textualize/rich>

Naive Bayes for Text Classification — Manning, Raghavan & Schutze (2008)